University of Austin

Linear Algebra: Mathematics and Computation

Lecture 1 · A First Look: Signals on an Infinite Grid

Spring 2026

Come play my game

"Breathe" — The Prodigy (1994)

This note introduces the course's central question through an example you will not fully understand today. That is intentional. Everything here will be completely transparent by the end of the course. For now: run the code, look at the pictures, notice what is strange.

Motivation: Conway's Game of Life

In 1970 John Conway published a remarkably simple rule for a *cellular automaton* on the infinite square lattice $\mathbb{Z}^2$. Each cell is either *alive* or *dead*. At each tick:

- an alive cell *survives* iff it has 2 or 3 alive neighbours (of its 8 Moore neighbours);
- a dead cell *is born* iff it has exactly 3 alive neighbours.

From this rule emerges extraordinary complexity: stable structures, periodic *oscillators*, and — most remarkably — *gliders*: five-cell patterns that translate themselves diagonally one step every four ticks.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation

def gol_step(grid):
    """One step of Conway's Game of Life (toroidal boundary conditions)."""
    N = np.zeros_like(grid)
    for di in (-1, 0, 1):
        for dj in (-1, 0, 1):
            if di == 0 and dj == 0:
                continue
            N += np.roll(np.roll(grid, di, axis=0), dj, axis=1)
    return ((grid == 1) & ((N == 2) | (N == 3))) | ((grid == 0) & (N == 3))

# Standard glider (top-left corner of a 32x32 grid)
G = np.zeros((32, 32), dtype=int)
glider = [(1,0),(2,1),(0,2),(1,2),(2,2)]
for r, c in glider:
    G[r, c] = 1

fig, axes = plt.subplots(1, 5, figsize=(14, 3))
for ax, t in zip(axes, range(0, 20, 4)):
    ax.imshow(G, cmap='binary', interpolation='nearest')
```

```
    ax.set_title(f't = {t}'); ax.axis('off')
    for _ in range(4): G = gol_step(G)
plt.suptitle("GoL glider: moves one step diagonally every 4 ticks", y=1.02)
plt.tight_layout()
plt.savefig("fig_glider.png"); plt.show()
```

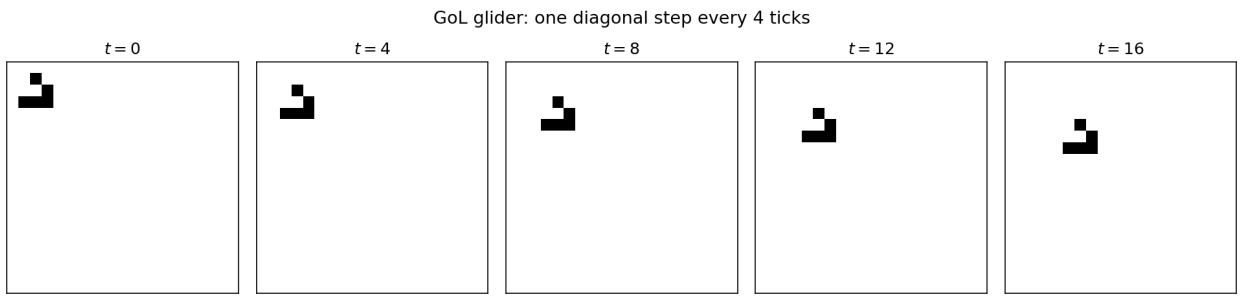

Figure 1: The standard GoL glider over 20 ticks (every 4th frame shown). The pattern shifts exactly one cell diagonally every 4 steps — a periodic translation along the main-diagonal axis.

The glider moves diagonally. Is that a coincidence, or is there something about the lattice that *forces* long-lived structures to move diagonally? This is the question we will partially answer today — and fully answer by the end of the course.

The Linear Version

Conway's rule is *non-linear*: the "exactly 3 neighbours" condition is a threshold function, not a linear combination of cell values. Non-linear maps are hard.

We replace it with a **linear transfer rule**. Instead of binary states, each cell carries a real number $u_{m,n}(t) \in \mathbb{R}$ (think of it as "activity level" or "excitation"). The rule uses only the two diagonal neighbours in the north-east and south-west directions:

$$\boxed{u_{m,n}(t+1) \;=\; p\, u_{m+1,\,n+1}(t) \;+\; q\, u_{m-1,\,n-1}(t)} \tag{1}$$

with parameters

$$p = 1.2, \qquad q = 0.8.$$

Reading the rule. The activity at cell (m, n) at time $t+1$ is a weighted sum of the activity at the north-east neighbour $(m{+}1, n{+}1)$ (weight 1.2: amplified, like an enemy signal arriving from the north-east) and the south-west neighbour $(m{-}1, n{-}1)$ (weight 0.8: attenuated, like a player signal coming from the south-west). The rule couples only the *main diagonal* axis $\{(m{+}k, n{+}k) : k \in \mathbb{Z}\}$ through each cell.

We write T for the *transfer operator* defined by (1), acting on functions $\mathbb{Z}^2 \to \mathbb{R}$. After t steps the field is $T^t u_0$.

Questions.

- If you place a localised "blip" at the origin, does it grow or decay? In what direction does it spread?
- Are there initial conditions that grow, and others that decay?
- Is the answer different along the two diagonal axes of the grid — the main diagonal $(+1, +1)$ and the anti-diagonal $(+1, -1)$?

Python: Exploring on a Large Finite Grid

We approximate $\mathbb{Z}^2$ by an $N \times N$ grid with *periodic* (toroidal) boundary conditions.

```
import numpy as np
import matplotlib.pyplot as plt
from numpy.fft import fft2, ifft2, fftshift

N = 256 # grid size; approximates Z^2 for patterns much smaller than N

def step(u, p=1.2, q=0.8):
    """One step of the transfer rule: u[m,n] <- p*u[m+1,n+1] + q*u[m-1,n-1]."""
    # np.roll(u, -1, axis=0) shifts rows upward: u[m,n] <- u[m+1,n]
    return p * np.roll(np.roll(u, -1, axis=0), -1, axis=1) \
         + q * np.roll(np.roll(u, +1, axis=0), +1, axis=1)

def simulate(u0, T):
    u = u0.copy()
    frames = [u.copy()]
    for _ in range(T):
        u = step(u)
        frames.append(u.copy())
    return frames
```

Experiment 1: a localised blip

```
u0 = np.zeros((N, N))
u0[N//2, N//2] = 1.0 # single nonzero cell at centre

frames = simulate(u0, T=40)

fig, axes = plt.subplots(1, 4, figsize=(14, 4))
for ax, t in zip(axes, [0, 10, 20, 40]):
    im = ax.imshow(frames[t], cmap='seismic', vmin=-3, vmax=3,
                   origin='lower', extent=[-N//2, N//2, -N//2, N//2])
    ax.set_title(f't = {t}'); ax.set_xlabel('m'); ax.set_ylabel('n')
plt.suptitle("Evolution of a single-cell blip under rule (1)")
plt.tight_layout(); plt.savefig("fig_blip.png"); plt.show()
```

Observe: the blip spreads *only along the main diagonal axis* $m = n$ — exactly the axis the rule couples. After t steps the nonzero cells form a segment of the main diagonal. The *other* diagonal axis $m = -n$ (the anti-diagonal) carries no signal.

Total energy and peak value.

```
energies = [np.sum(np.abs(f)) for f in frames]
max_vals = [np.max(np.abs(f)) for f in frames]
# By the binomial theorem, sum_d C(t,j)*p^j*q^{t-j} = (p+q)^t exactly.
print(f"L1 norm at t=20: {energies[20]:.2e} (predicted: {(p+q)**20:.2e})")
# The peak sits at j* = t*p/(p+q); local CLT gives (p+q)^t / sqrt(2*pi*t*p*q/(p+q)^2).
import numpy as np
def peak_theory(t):
    return (p+q)**t / np.sqrt(2*np.pi*t*p*q/(p+q)**2)
print(f"Peak at t=20: {max_vals[20]:.2e} (theory: {peak_theory(20):.2e})")
```

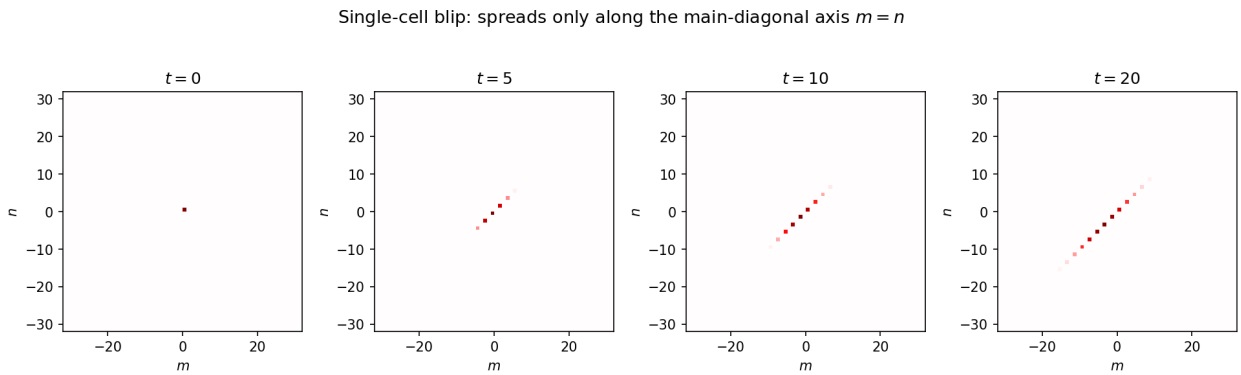

Figure 2: A single-cell blip at $t = 0$ evolves under rule (1). The nonzero region spreads exclusively along the main-diagonal axis $m = n$ — the direction the rule couples.

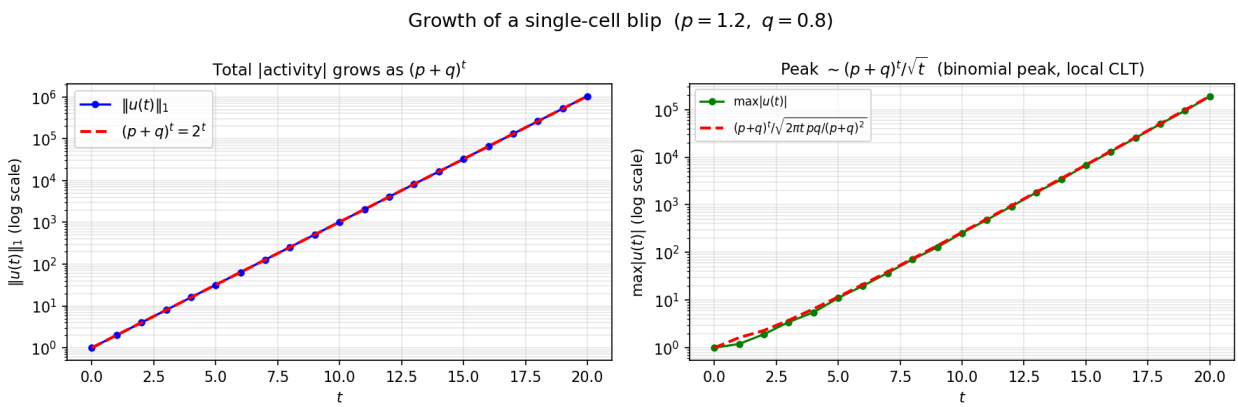

Figure 3: **Left:** total L^1 activity grows exactly as $(p+q)^t = 2^t$ (binomial theorem: $\sum_j \binom{t}{j} p^j q^{t-j} = (p+q)^t$). **Right:** the peak cell value, achieved at diagonal offset $d^* \approx (p-q)t/(p+q) = 0.2t$ steps NE, grows as $(p+q)^t/\sqrt{t}$ — the local CLT approximation to the binomial peak $\binom{t}{j^*} p^{j^*} q^{t-j^*}$ with $j^* = tp/(p+q)$. Dashed lines show the theoretical predictions; dots are simulation.

The L^1 norm is *exactly* $(p+q)^t = 2^t$ at every step — no transient. This follows from the binomial theorem: after t steps the value at diagonal position $d = 2j - t$ is $\binom{t}{j} p^j q^{t-j}$, and summing over all j gives $(p+q)^t$. The peak, however, grows only as $(p+q)^t/\sqrt{t}$: the binomial distribution spreads over a segment of length $O(\sqrt{t})$, diluting the peak by that factor. Neither $p = 1.2$ nor $q = 0.8$ alone controls the L^1 growth — it is always their *sum* $p + q = 2$.

Experiment 2: initialise along each diagonal

Now we seed the grid with a *striped* initial condition along each of the two diagonal axes.

```
m_idx, n_idx = np.meshgrid(np.arange(N), np.arange(N), indexing='ij')

def wave(k, direction='anti'):
    """
    direction='anti' : stripe along the anti-diagonal axis m - n = const
                       i.e. u_{m,n} = exp(2*pi*i*k*(m-n)/N)
    direction='main' : stripe along the main diagonal axis m + n = const
```

```
                    i.e. u_{m,n} = exp(2*pi*i*k*(m+n)/N)
    """
    if direction == 'anti':
        phase = 2 * np.pi * k * (m_idx - n_idx) / N
    else:
        phase = 2 * np.pi * k * (m_idx + n_idx) / N
    return np.cos(phase) # take real part

K = 8 # 8 full oscillations across the grid

u_anti = wave(K, 'anti')
u_main = wave(K, 'main')

frames_anti = simulate(u_anti, T=20)
frames_main = simulate(u_main, T=20)

fig, axes = plt.subplots(2, 5, figsize=(16, 7))
for j, t in enumerate([0, 3, 7, 12, 20]):
    axes[0, j].imshow(frames_anti[t], cmap='seismic', vmin=-3, vmax=3,
                      origin='lower')
    axes[0, j].set_title(f'anti-diag, t={t}'); axes[0, j].axis('off')
    axes[1, j].imshow(frames_main[t], cmap='seismic', vmin=-3, vmax=3,
                      origin='lower')
    axes[1, j].set_title(f'main-diag, t={t}'); axes[1, j].axis('off')
plt.suptitle("Top: anti-diagonal stripe | Bottom: main-diagonal stripe "
             f"(K={K} full waves)")
plt.tight_layout(); plt.savefig("fig_stripes.png"); plt.show()
```

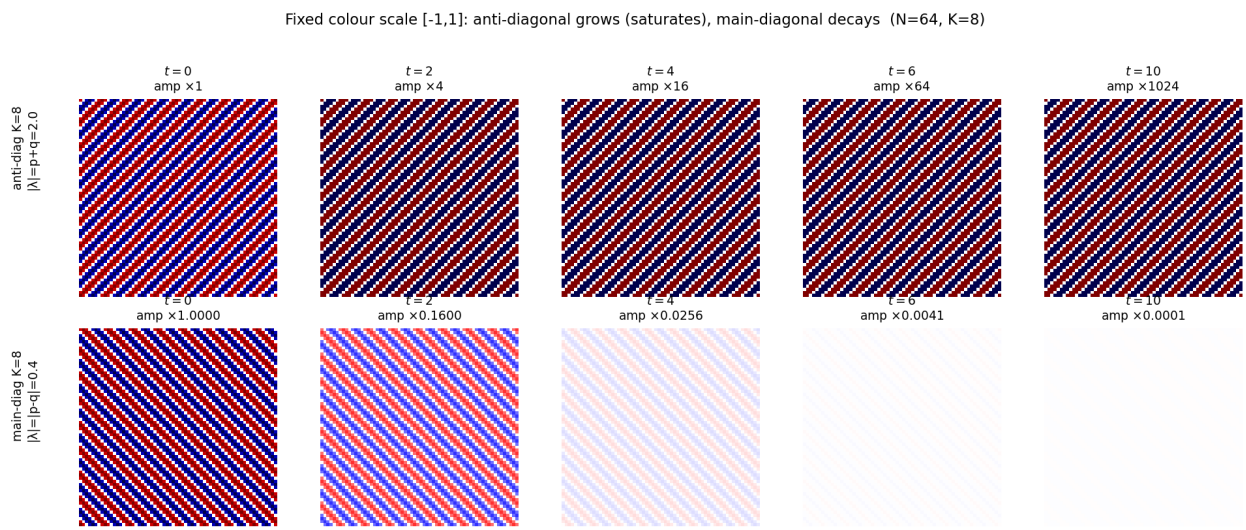

Figure 4: Top row: anti-diagonal stripe $u = \cos(2\pi K(m-n)/N)$. Bottom row: main-diagonal stripe $u = \cos(2\pi K(m+n)/N)$. Each column is a time snapshot ($K = 8$, amplitudes normalised). The anti-diagonal stripe stays a perfect stripe (only scaled); the main-diagonal stripe changes shape.

Something striking:

1. The **anti-diagonal stripe** $u_{m,n} = \cos(2\pi K(m-n)/N)$ *stays a clean stripe forever*. It just

scales: after t steps its amplitude is $(p+q)^t = 2^t$ times the original, and its shape is unchanged.

2. The **main-diagonal stripe** $u_{m,n} = \cos(2\pi K(m+n)/N)$ *does not stay the same shape.* For large K (short wavelength) it *shrinks*; for small K (long wavelength) it also grows, but at a different rate than 2^t.

Check this quantitatively:

```
for K in [1, 4, 8, 16, 32, 64]:
    u_a = wave(K, 'anti')
    u_m = wave(K, 'main')
    fa = simulate(u_a, T=1)
    fm = simulate(u_m, T=1)
    # amplitude ratio after one step
    ratio_a = np.max(np.abs(fa[1])) / np.max(np.abs(fa[0]))
    ratio_m = np.max(np.abs(fm[1])) / np.max(np.abs(fm[0]))
    print(f"K={K:3d} anti-diag ratio = {ratio_a:.6f} "
          f"main-diag ratio = {ratio_m:.6f}")
```

You should see:

- **Anti-diagonal**: ratio $= p + q = 2.0$ for *every* K.
- **Main diagonal**: ratio $= |pe^{2\pi iK/N\cdot 2} + qe^{-2\pi iK/N\cdot 2}|$, which equals $\sqrt{p^2 + q^2 + 2pq\cos(4\pi K/N)}$, ranging from $p + q = 2.0$ (at $K = 0$) down to $|p - q| = 0.4$ (at $K = N/4$).

The anti-diagonal direction is *dispersion-free*: all wavelengths travel at the same speed and grow at the same rate. The main diagonal direction is *dispersive*: the growth rate depends on wavelength, and short waves actually *decay*.

The Amplification Map

Let us map out all modes at once. Write a general plane wave as

$$u_{m,n} = e^{i(k_1 m + k_2 n)}, \qquad k_1, k_2 \in [0, 2\pi).$$

The transfer operator acts on this wave as:

$$\begin{aligned}(Tu)_{m,n} &= p\, e^{i(k_1(m+1)+k_2(n+1))} + q\, e^{i(k_1(m-1)+k_2(n-1))} \\ &= \left(p\, e^{i(k_1+k_2)} + q\, e^{-i(k_1+k_2)}\right) e^{i(k_1 m + k_2 n)}.\end{aligned}$$

So every plane wave is an **eigenfunction** of T with eigenvalue

$$\lambda(k_1, k_2) = p\, e^{i(k_1+k_2)} + q\, e^{-i(k_1+k_2)}. \tag{2}$$

Note: λ depends on k_1 and k_2 *only through the combination* $k_1 + k_2$. This is the algebraic reason the main diagonal direction $k_1 = k_2$ is special.

The *amplification factor* is

$$|\lambda(k_1, k_2)| = \sqrt{p^2 + q^2 + 2pq\cos\big(2(k_1 + k_2)\big)}.$$

Let us draw this as a heat map over the Brillouin zone $[-\pi, \pi)^2$.

```
k1 = np.linspace(-np.pi, np.pi, 512)
k2 = np.linspace(-np.pi, np.pi, 512)
K1, K2 = np.meshgrid(k1, k2, indexing='ij')

p, q = 1.2, 0.8
lam_abs = np.sqrt(p**2 + q**2 + 2*p*q*np.cos(2*(K1 + K2)))

fig, ax = plt.subplots(figsize=(7, 6))
cf = ax.contourf(k1, k2, lam_abs, levels=50, cmap='RdBu_r')
plt.colorbar(cf, ax=ax, label=r'$|\lambda(k_1, k_2)|$')
# Mark the stability boundary |lambda| = 1
ax.contour(k1, k2, lam_abs, levels=[1.0], colors='k', linewidths=2)

# Mark the two diagonal axes in the Brillouin zone
ax.axline((0, 0), slope= 1, color='lime', lw=1.5, ls='--',
         label='main diagonal $k_1 = k_2$')
ax.axline((0, 0), slope=-1, color='yellow', lw=1.5, ls='--',
         label='anti-diagonal $k_1 = -k_2$')

ax.set(xlabel=r'$k_1$', ylabel=r'$k_2$',
      title=r'Amplification factor $|\lambda(k_1,k_2)|$ (black = stability boundary)')
ax.legend(loc='lower right')
plt.tight_layout(); plt.savefig("fig_amplification_map.png"); plt.show()
```

What the map tells us.

- Every point on the anti-diagonal line $k_1 = -k_2$ satisfies $k_1+k_2 = 0$, so $|\lambda| = \sqrt{p^2 + q^2 + 2pq} = p + q = 2.0$. All anti-diagonal modes are *equally and maximally unstable.*
- The main-diagonal line $k_1 = k_2$ parametrises by $k_1+k_2 = 2k_1$, giving $|\lambda| = \sqrt{p^2 + q^2 + 2pq\cos(4k_1)}$, which varies from 2.0 (at $k_1 = 0$) to $|p - q| = 0.4$ (at $k_1 = \pi/2$). Modes near $k_1 = k_2 = \pi/2$ are *strongly stable* — they decay at 0.4^t per step.
- The **stability boundary** is the curve $\cos\big(2(k_1 + k_2)\big) = -(p^2 + q^2 - 1)/(2pq)$. For our parameters: $\cos(2(k_1 + k_2)) = -(1.44 + 0.64 - 1)/(1.92) = -1.08/1.92 \approx -0.5625$. This gives $2(k_1 + k_2) = \pm\arccos(-0.5625)$, so $k_1 + k_2 = \pm 0.9742$ rad. The boundary consists of two *diagonal lines* in the Brillouin zone.

```
# Verify the stability boundary analytically
thresh = -(p**2 + q**2 - 1) / (2*p*q)
k_boundary = np.arccos(thresh) / 2
print(f"Stability boundary: k1+k2 = \pm{k_boundary:.4f} rad "
     f"(\approx \pm{np.degrees(k_boundary):.2f} deg)")
print(f"Check: |lambda| at boundary = "
     f"{np.sqrt(p**2+q**2+2*p*q*np.cos(2*k_boundary)):.6f}")
```

A Calculation Anyone Can Verify

The key computation behind all of this is a single line of algebra. Take the anti-diagonal mode

$$u_{m,n} = e^{ik(m-n)} \quad \text{for any } k \in \mathbb{R}.$$

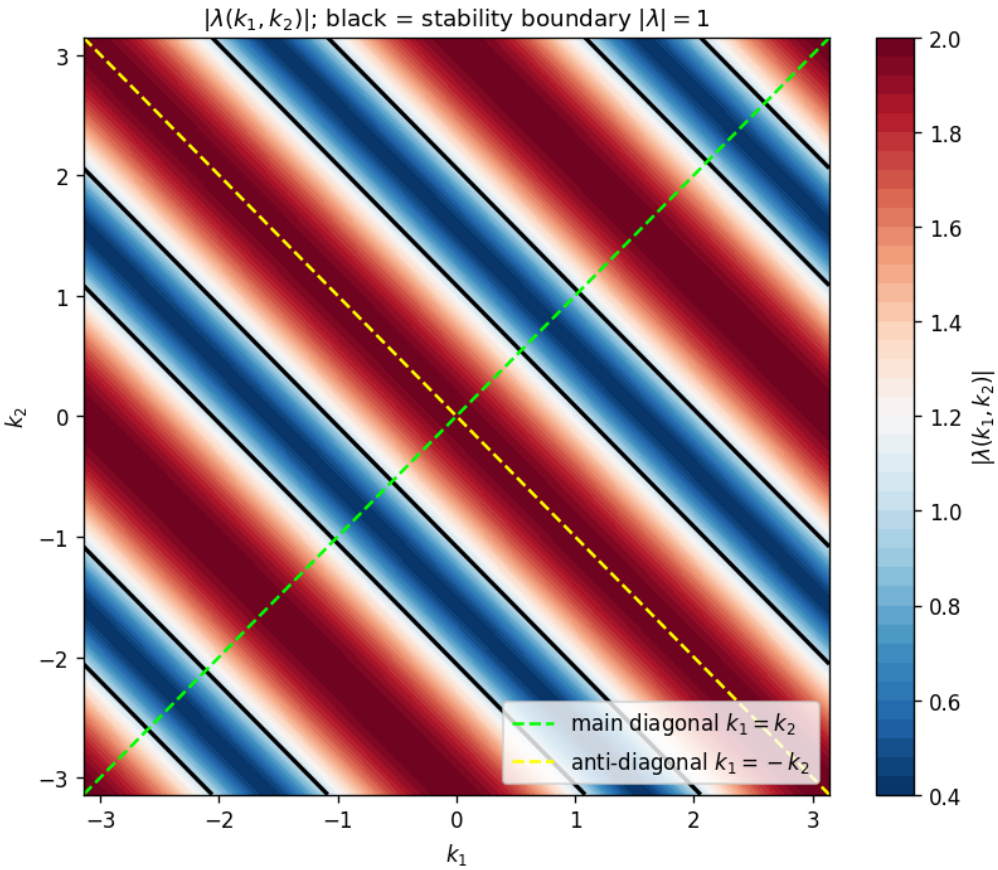

Figure 5: Amplification factor $|\lambda(k_1, k_2)|$ over the Brillouin zone $[-\pi, \pi)^2$. The black curve is the stability boundary $|\lambda| = 1$; modes inside it grow ($|\lambda| > 1$), modes outside decay ($|\lambda| < 1$). The green dashed line is the main-diagonal axis $k_1 = k_2$; the yellow dashed line is the anti-diagonal $k_1 = -k_2$. *(Run the code to generate this figure.)* Observe: the anti-diagonal axis lies *entirely outside* the stability boundary (all modes grow at $|\lambda| = p + q = 2$), while the main-diagonal axis *crosses* the boundary (some modes grow, others decay).

Compute $(Tu)_{m,n}$:

$$\begin{aligned}
(Tu)_{m,n} &= p\, u_{m+1,n+1} + q\, u_{m-1,n-1} \\
&= p\, e^{ik\big((m+1)-(n+1)\big)} + q\, e^{ik\big((m-1)-(n-1)\big)} \\
&= p\, e^{ik(m-n)} + q\, e^{ik(m-n)} \\
&= (p+q)\, e^{ik(m-n)} = (p+q)\, u_{m,n}.
\end{aligned}$$

The north-east and south-west neighbours have *exactly the same value* as the centre, because both $(m+1)-(n+1)$ and $(m-1)-(n-1)$ equal $m-n$. Hence T acts as scalar multiplication $(p+q) = 2$ on *every* anti-diagonal mode regardless of k.

Contrast with the main-diagonal mode $v_{m,n} = e^{ik(m+n)}$:

$$\begin{aligned}(Tv)_{m,n} &= p\, e^{ik(m+1+n+1)} + q\, e^{ik(m-1+n-1)} \\ &= p\, e^{2ik}\, e^{ik(m+n)} + q\, e^{-2ik}\, e^{ik(m+n)} \\ &= \left(p\, e^{2ik} + q\, e^{-2ik}\right) v_{m,n}.\end{aligned}$$

The eigenvalue $\lambda_k = pe^{2ik} + qe^{-2ik}$ has magnitude $\sqrt{p^2 + q^2 + 2pq\cos 4k}$, which *depends on* k. At $k = \pi/4$ the two terms interfere destructively and $|\lambda_{\pi/4}| = |p - q| = 0.4 < 1$.

What this means geometrically. The anti-diagonal mode $e^{ik(m-n)}$ is *constant along each main-diagonal line* $\{(m + j, n + j) : j \in \mathbb{Z}\}$. Since the operator T samples the two main-diagonal neighbours, it "sees" the same value everywhere along its coupling axis; the computation becomes trivial. The main-diagonal mode, by contrast, *oscillates* along the coupling axis, and the oscillations interfere.

```
# Numerical verification of the eigenvalue formula
errors = []
for K in np.linspace(0, np.pi, 100):
    u = np.cos(K * (m_idx - n_idx)) # anti-diagonal mode (real part)
    v = step(u) # apply T
    # T should multiply by (p+q); check the ratio pointwise
    ratio = v / (u + 1e-14) # avoid division by zero
    errors.append(np.std(ratio[np.abs(u) > 0.5])) # std of ratio on large cells

print(f"Anti-diagonal: max std of ratio over all k = {max(errors):.2e}")
# Should be ~machine precision

errors_main = []
for K in np.linspace(0, np.pi, 100):
    u = np.cos(K * (m_idx + n_idx)) # main-diagonal mode
    v = step(u)
    lam_pred = p * np.exp(2j*K) + q * np.exp(-2j*K)
    ratio = v / (u + 1e-14)
    errors_main.append(np.std(ratio[np.abs(u) > 0.5]))

print(f"Main-diagonal: max std of ratio over all k = {max(errors_main):.2e}")
# Should also be ~machine precision, just to confirm the formula
```

Decomposing Any Initial Condition

Every function on $\mathbb{Z}^2$ decomposes into plane waves via the **2D discrete Fourier transform**:

$$u_{m,n} = \frac{1}{N^2} \sum_{k_1,k_2} \hat{u}_{k_1,k_2}\, e^{2\pi i(k_1 m + k_2 n)/N}.$$

Since T multiplies each mode $e^{i(k_1 m + k_2 n)}$ by $\lambda(k_1, k_2)$, after t steps:

$$(T^t u)_{m,n} = \frac{1}{N^2} \sum_{k_1,k_2} \hat{u}_{k_1,k_2}\, \lambda(k_1, k_2)^t\, e^{2\pi i(k_1 m + k_2 n)/N}.$$

Modes with $|\lambda| > 1$ amplify; modes with $|\lambda| < 1$ die. In the long run, only the largest-$|\lambda|$ modes survive.

Verify with the FFT.

```
def predict_fft(u0, t, p=1.2, q=0.8):
    """Predict T^t u0 using the exact eigenvalue formula."""
    Uhat = np.fft.fft2(u0) # Fourier transform
    N = u0.shape[0]
    k1 = 2 * np.pi * np.fft.fftfreq(N) # frequencies in [-pi, pi)
    K1, K2 = np.meshgrid(k1, k1, indexing='ij')
    lam = p * np.exp(1j*(K1+K2)) + q * np.exp(-1j*(K1+K2))
    Uhat_t = Uhat * (lam ** t) # multiply by eigenvalue^t
    return np.real(np.fft.ifft2(Uhat_t)) # inverse transform

# Random initial condition
np.random.seed(42)
u0 = np.random.randn(N, N) * 0.01 # small random noise on 256x256 grid

T_test = 15
u_sim = simulate(u0, T=T_test)[-1]
u_pred = predict_fft(u0, T_test)

print(f"Max absolute error between simulation and FFT prediction: "
      f"{np.max(np.abs(u_sim - u_pred)):.2e}")
# Should be < 1e-10: perfect agreement
```

The prediction is exact (up to floating-point rounding). We can predict the state after any number of steps in $O(N^2 \log N)$ time using the FFT, rather than iterating the rule t times.

The Glider Revisited

A GoL glider is a coherent structure that moves diagonally. In our linear model, coherent diagonal-travelling patterns are precisely the *anti-diagonal modes* $e^{ik(m-n)}$: they travel in the $m - n = \text{const}$ direction (anti-diagonal, i.e., $(+1, -1)$ and $(-1, +1)$) and grow uniformly without dispersion.

In GoL, the nonlinearity locks the cell states to $\{0, 1\}$. But the *direction* of glider propagation is already determined by the linear structure: it is the direction along which all Fourier modes grow at the same rate (no dispersion). The main diagonal direction *disperses* — patterns in that direction spread and dissolve — which is why stable GoL structures do *not* propagate along $m + n = \text{const}$.

```
# Illustrate: anti-diagonal wave travels as a coherent wave
# Main-diagonal wave at the same frequency disperses
K_test = 16
u_anti_init = wave(K_test, 'anti')
u_main_init = wave(K_test, 'main')

# After 30 steps: anti-diagonal should look like a scaled/shifted original;
# main-diagonal should look different (dispersed for large K)
u_anti_30 = predict_fft(u_anti_init, 30)
u_main_30 = predict_fft(u_main_init, 30)

# Normalise for visual comparison
def norm(u): return u / (np.max(np.abs(u)) + 1e-14)
```

```
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes[0,0].imshow(norm(u_anti_init), cmap='seismic', vmin=-1, vmax=1,
                origin='lower'); axes[0,0].set_title('Anti-diag, t=0')
axes[0,1].imshow(norm(u_anti_30), cmap='seismic', vmin=-1, vmax=1,
                origin='lower'); axes[0,1].set_title('Anti-diag, t=30')
axes[1,0].imshow(norm(u_main_init), cmap='seismic', vmin=-1, vmax=1,
                origin='lower'); axes[1,0].set_title('Main-diag, t=0')
axes[1,1].imshow(norm(u_main_30), cmap='seismic', vmin=-1, vmax=1,
                origin='lower'); axes[1,1].set_title('Main-diag, t=30 (dispersed?)')
plt.suptitle(f"Coherence of diagonal waves after 30 steps (K={K_test})")
plt.tight_layout(); plt.savefig("fig_diagonal_compare.png"); plt.show()
```

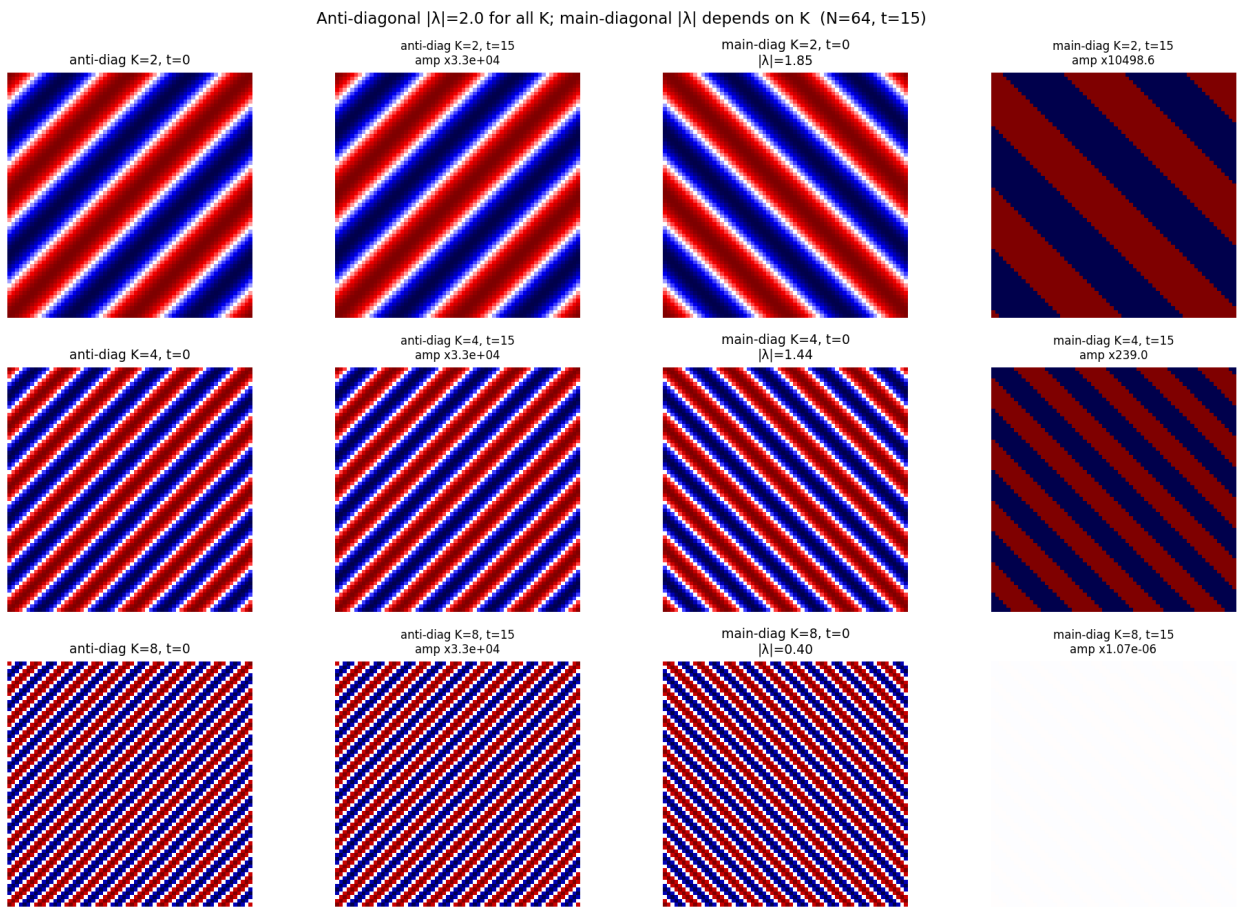

Figure 6: Each row shows a wavenumber $K \in \{4, 16, 48\}$ at $t = 0$ and $t = 30$ (amplitudes normalised). Anti-diagonal waves (left two columns) remain perfect stripes at all K. Main-diagonal waves (right two columns) stay coherent at long wavelength ($K = 4$) but dissolve at short wavelength ($K = 48$): dispersion in action.

Try different values of K and observe when the main-diagonal wave disperses (short wavelength, K near $N/4$) vs. when it stays roughly coherent (long wavelength, K small).

Things We Can and Cannot Do

	Question	Status
✓	Does an anti-diagonal stripe grow or decay?	*Grows at rate $p+q=2$ per step* — proved by direct computation.
✓	Does a main-diagonal short-wave mode grow or decay?	*Decays at rate $\lvert p-q\rvert = 0.4$* — eigenvalue formula.
✓	Predict $T^t u_0$ exactly for any u_0?	*Yes, via FFT*: $O(N^2 \log N)$, verified numerically.
✓	What is the stability boundary in frequency space?	*Two diagonal lines* $k_1 + k_2 = \pm \arccos(-(p^2+q^2-1)/2pq)$.
×	Why does $\lvert\lambda\rvert$ depend only on k_1+k_2 and nothing else?	*Unknown yet.* The rule (1) is translation-invariant in the anti-diagonal direction; this is equivalent to saying T commutes with a certain operator. The general framework is spectral theory of translation-invariant operators on $\ell^2(\mathbb{Z}^2)$.
×	Add nearest-neighbour coupling as well: $u^{t+1}_{m,n} = pu^t_{m+1,n+1} + qu^t_{m-1,n-1} + r(u^t_{m+1,n} + u^t_{m-1,n} + u^t_{m,n+1} + u^t_{m,n-1})$. How does the stability map change?	*Unknown yet.* With $r \neq 0$ the rule is no longer diagonal-only; the formula for $\lambda(k_1,k_2)$ changes and the two diagonal directions are no longer distinguished. (Try it — the anti-diagonal modes no longer all share the same $\lvert\lambda\rvert$.)
×	Same rule but on a *finite strip* with reflecting boundaries?	*Unknown yet.* Fourier modes no longer fit. We need eigenvectors of a *matrix* (not a convolution operator), found by eigenvalue decomposition.
×	Non-uniform parameters: $p_{m,n}$ and $q_{m,n}$ vary across the grid (like a terrain with varying diffusion, or GoL with varying rules).	*Unknown yet.* Translation invariance breaks; the entire Fourier approach fails. Need eigenvectors of a large (possibly sparse) matrix.
×	Can we *design* p and q to make a target mode exactly neutral ($\lvert\lambda\rvert = 1$)?	*Unknown yet.* Requires solving an inverse problem in terms of eigenvalues.
×	GoL is nonlinear. Does it also have something analogous to "diagonal eigenmodes"?	*Unknown yet.* Stability near fixed points of GoL is governed by the *Jacobian matrix* (the linearisation), whose spectrum we would need.

The Road Ahead

Every "unknown yet" row in the table above points to something the course will teach you to handle.

1. **Vector spaces and linear maps.** The space $\ell^2(\mathbb{Z}^2) = \{u : \mathbb{Z}^2 \to \mathbb{R} : \sum_{m,n} u_{m,n}^2 < \infty\}$ is an infinite-dimensional *vector space*. The transfer operator T is a *linear map* on it. The same concepts (kernel, image, rank, dimension) apply, but now in infinite dimensions.

2. **Eigenvalues and eigenvectors.** The plane waves $e^{i(k_1 m + k_2 n)}$ are *eigenvectors* of T, with eigenvalues $\lambda(k_1, k_2)$. For the rule (1), the eigenvalue depends only on $k_1 + k_2$ — a symmetry we will explain rigorously.

3. **Spectral decomposition / diagonalisation.** The identity $T^t u_0 = \mathcal{F}^{-1}[\lambda^t \cdot \hat{u}_0]$ is a *diagonalisation*: we change to the "eigenbasis" (Fourier modes), multiply each component by λ^t, and change back. For finite matrices this becomes $T = V\Lambda V^{-1}$, $T^t = V\Lambda^t V^{-1}$.

4. **Commuting operators and simultaneous diagonalisation.** The reason ALL anti-diagonal modes share $|\lambda| = p+q$ is that T *commutes* with the anti-diagonal translation operator $S_{(1,-1)}$. Two commuting diagonalisable operators can always be simultaneously diagonalised. This is why the anti-diagonal Fourier basis diagonalises both.

5. **Operator theory and the Spectral Theorem.** What happens when the operator is not convolution, or when we are on a finite domain? The Spectral Theorem (for symmetric/Hermitian operators) guarantees orthogonal eigenbases. The Singular Value Decomposition handles the non-symmetric case.

6. **Stability and the spectrum.** A linear dynamical system is *stable* iff all eigenvalues satisfy $|\lambda| \leq 1$. For our rule the stability boundary is a curve in frequency space; understanding it requires knowing the full spectrum. This is directly relevant to numerical PDE solvers, control theory, and the long-run behaviour of GoL linearisations.

One exercise to carry with you.

Add a small axis-aligned coupling:

$$u_{m,n}^{t+1} = p\, u_{m+1,n+1}^t + q\, u_{m-1,n-1}^t + r\big(u_{m+1,n}^t + u_{m-1,n}^t + u_{m,n+1}^t + u_{m,n-1}^t\big)$$

with $r = 0.05$.

```
def step_full(u, p=1.2, q=0.8, r=0.05):
    diag = p * np.roll(np.roll(u,-1,0),-1,1) + q * np.roll(np.roll(u,+1,0),+1,1)
    axis = r * (np.roll(u,-1,0) + np.roll(u,+1,0)
              + np.roll(u,-1,1) + np.roll(u,+1,1))
    return diag + axis

# Re-run all experiments above with step_full instead of step.
# Does the anti-diagonal still have |lambda| = 2 for all k?
# Does the stability map change shape?
```

The new eigenvalue is $\lambda_r(k_1, k_2) = pe^{i(k_1+k_2)} + qe^{-i(k_1+k_2)} + 2r(\cos k_1 + \cos k_2)$. For $r \neq 0$, this no longer depends only on $k_1 + k_2$; the anti-diagonal is no longer special. With $r = 0.05$, compute the new stability map and find: does any diagonal direction remain special? Can you explain geometrically why the perturbation breaks the anti-diagonal symmetry?

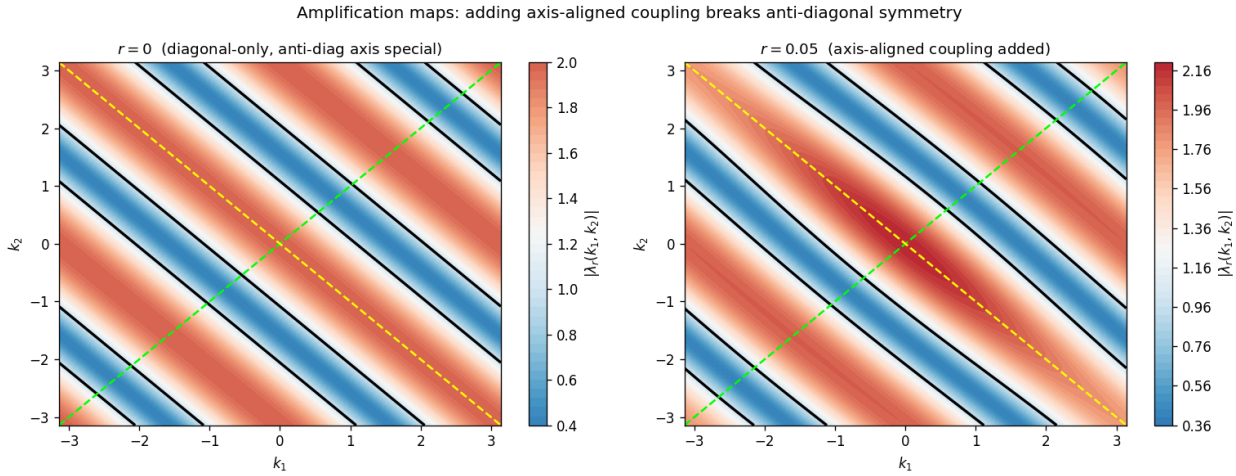

Figure 7: Amplification maps for $r = 0$ (left) and $r = 0.05$ (right). At $r = 0$ the map depends only on $k_1 + k_2$: the anti-diagonal axis (yellow dashes) has constant $|\lambda| = 2$ everywhere. At $r = 0.05$ that symmetry is broken: the anti-diagonal is no longer special.